

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 18
“MESIN ABSTRAK”



DISUSUN OLEH:
NADIFA AZKHIA SYARIF
109082530002
S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Mesin abstrak merupakan suatu model komputasi yang digunakan untuk merepresentasikan suatu sistem melalui sekumpulan data dan operasi yang dapat dilakukan terhadap data tersebut. Menurut Goodrich, Tamassia, dan Goldwasser (2014), konsep abstraksi memungkinkan programmer untuk berfokus pada fungsi dan perilaku suatu objek tanpa harus memperhatikan detail implementasinya. Dengan adanya abstraksi, suatu permasalahan dapat diselesaikan secara lebih terstruktur dan mudah dipahami.

Salah satu bentuk penerapan mesin abstrak dalam pemrograman adalah penggunaan Abstract Data Type (ADT) atau tipe data abstrak. ADT merupakan spesifikasi mengenai data dan operasi yang dapat dilakukan terhadap data tersebut tanpa menjelaskan bagaimana data tersebut direpresentasikan di dalam memori komputer. Melalui ADT, sebuah objek dapat dimodelkan sesuai kebutuhan sehingga mempermudah proses pengembangan perangkat lunak. Pada praktikum ini, kartu domino direpresentasikan sebagai sebuah ADT yang memiliki atribut dan operasi tertentu.

Kartu domino merupakan objek yang terdiri atas dua sisi yang masing-masing memiliki nilai tertentu. Dalam implementasi program, kartu domino dapat direpresentasikan menggunakan struktur data yang menyimpan informasi mengenai kedua sisi kartu, nilai kartu, dan status kartu balak. Kumpulan kartu domino kemudian disimpan dalam suatu struktur data yang memungkinkan operasi seperti pengacakan, pengambilan, pencarian, dan pemeriksaan nilai kartu dilakukan secara efisien. Menurut Cormen et al. (2022), pemilihan struktur data yang tepat akan memengaruhi efisiensi algoritma yang digunakan dalam pengolahan data.

Struktur data array merupakan salah satu struktur data yang sering digunakan untuk menyimpan sekumpulan elemen dengan tipe yang sama. Array memiliki keunggulan dalam hal kemudahan akses data berdasarkan indeks serta penggunaan memori yang relatif sederhana. Pada implementasi mesin domino, array digunakan untuk menyimpan satu set kartu domino yang terdiri atas 28 kartu. Dengan memanfaatkan array, proses pengelolaan kartu dapat dilakukan secara sistematis dan terorganisir.

Selain pengelolaan data, algoritma juga memegang peranan penting dalam pengembangan program. Menurut Knuth (1997), algoritma adalah serangkaian langkah logis dan teratur yang digunakan untuk menyelesaikan suatu masalah. Operasi seperti pengacakan kartu, pengambilan kartu dari tumpukan, pencarian kartu yang sesuai, serta pemeriksaan nilai kartu merupakan contoh penerapan algoritma dalam mesin domino. Algoritma yang baik mampu menghasilkan solusi yang benar dengan penggunaan sumber daya yang efisien.

Pada praktikum ini juga digunakan konsep mesin karakter. Mesin karakter merupakan suatu model yang digunakan untuk membaca karakter dalam sebuah rangkaian teks secara berurutan dari awal hingga akhir. Setiap karakter dibaca satu per satu menggunakan operasi dasar tertentu, seperti memulai pembacaan, membaca karakter saat ini, berpindah ke karakter berikutnya, dan memeriksa apakah pembacaan telah mencapai akhir data. Konsep ini banyak digunakan dalam pemrosesan teks, compiler, interpreter, maupun berbagai aplikasi pengolahan bahasa.

Pemrosesan karakter memungkinkan komputer melakukan berbagai analisis terhadap teks yang diberikan. Analisis tersebut dapat berupa perhitungan jumlah karakter, pencarian karakter tertentu, penghitungan frekuensi kemunculan huruf, hingga pencarian pola kata tertentu. Menurut Jurafsky dan Martin (2023), analisis frekuensi karakter dan kata merupakan salah satu teknik dasar dalam pengolahan bahasa alami (Natural Language Processing) yang digunakan untuk memperoleh informasi dari data berbentuk teks.

Dengan memanfaatkan konsep mesin abstrak, tipe data abstrak, struktur data, algoritma, serta mesin karakter, berbagai permasalahan yang berkaitan dengan pengelolaan kartu domino dan pengolahan teks dapat diselesaikan secara sistematis. Oleh karena itu, pemahaman terhadap konsep-konsep tersebut menjadi dasar penting dalam pengembangan program yang efektif dan efisien.

SOAL LATIHAN

1.

Source Code:

```
package main

import "fmt"

type Domino struct {
    Sisi1 int
    Sisi2 int
    Nilai int
    Balak bool
}

type Dominoes struct {
    Kartu [28]Domino
    Jumlah int
}

func main() {
    var kartu Domino

    kartu.Sisi1 = 3
    kartu.Sisi2 = 3
    kartu.Nilai = 6
    kartu.Balak = true

    fmt.Println(kartu)
}
```

Output:

{3 3 6 true}

b.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    Sisi1 int
    Sisi2 int
}

type Dominoes struct {
    Kartu  [28]Domino
    Jumlah int
}

func kocokKartu(d *Dominoes) {
    rand.Seed(time.Now().UnixNano())

    for i := range d.Kartu {
        j := rand.Intn(len(d.Kartu))
        d.Kartu[i], d.Kartu[j] = d.Kartu[j], d.Kartu[i]
    }
}

func main() {
    var deck Dominoes

    kocokKartu(&deck)

    fmt.Println("Kartu berhasil dikocok")
}
```

Output:

Kartu berhasil dikocok

c.

Source Code:

```
package main

import "fmt"

type Domino struct {
    Sisi1 int
    Sisi2 int
}

type Dominoes struct {
    Kartu  [28]Domino
    Jumlah int
}

func ambilKartu(d *Dominoes) Domino {

    if d.Jumlah == 0 {
        return Domino{}
    }

    d.Jumlah--
    return d.Kartu[d.Jumlah]
}

func main() {

    var deck Dominoes

    deck.Kartu[0] = Domino{2, 5}
    deck.Jumlah = 1

    kartu := ambilKartu(&deck)

    fmt.Println(kartu)
}
```

Output:
{2 5}

d.

Source Code:

```
package main

import "fmt"

type Domino struct {
    Sisi1 int
    Sisi2 int
}

func gambarKartu(d Domino, suit int) int {

    if suit == 1 {
        return d.Sisi1
    }

    return d.Sisi2
}

func main() {

    kartu := Domino{3, 6}

    fmt.Println(gambarKartu(kartu, 1))
    fmt.Println(gambarKartu(kartu, 2))
}
```

Output:

3
6

Source Code:

```
package main

import "fmt"
```

```

type Domino struct {
    Sisi1 int
    Sisi2 int
    Nilai int
}

func nilaiKartu(d Domino) int {
    return d.Nilai
}

func main() {

    kartu := Domino{2, 5, 7}

    fmt.Println(nilaiKartu(kartu))
}

```

Output:

7

Deskripsi Program:

Program ini bertujuan untuk mengimplementasikan operasi dasar pada mesin abstrak domino. Program diawali dengan pembuatan tipe data untuk merepresentasikan kartu domino beserta satu set kartu domino. Selain itu, program menyediakan beberapa operasi dasar seperti mengocok kartu, mengambil kartu dari tumpukan, menampilkan gambar atau sisi kartu, serta menghitung nilai kartu. Operasi-operasi tersebut menjadi dasar dalam pengelolaan data kartu domino pada berbagai permainan yang menggunakan kartu domino.

2.

Source Code:

```

package main

import "fmt"

type Domino struct {
    Sisi1 int
    Sisi2 int
}

type Dominoes struct {
    Kartu [28]Domino
    Jumlah int
}

```

```

}

func ambilKartu(d *Dominoes) Domino {

    d.Jumlah--
    return d.Kartu[d.Jumlah]
}

func galiKartu(deck *Dominoes, target Domino) {

    for deck.Jumlah > 0 {

        kartu := ambilKartu(deck)

        if kartu.Sisi1 == target.Sisi1 ||
           kartu.Sisi2 == target.Sisi1 ||
           kartu.Sisi1 == target.Sisi2 ||
           kartu.Sisi2 == target.Sisi2 {

            fmt.Println("Kartu ditemukan")
            fmt.Println(kartu)
            return
        }
    }

    fmt.Println("Tidak ditemukan")
}

func main() {

    var deck Dominoes

    deck.Kartu[0] = Domino{1, 4}
    deck.Kartu[1] = Domino{5, 6}
    deck.Jumlah = 2

    target := Domino{4, 2}

    galiKartu(&deck, target)
}

```

Output:
Kartu ditemukan
{1 4}

B.

Source Code:

```
package main

import "fmt"

type Domino struct {
    Nilai int
}

func sepasangKartu(a Domino, b Domino) bool {

    return a.Nilai+b.Nilai == 12
}

func main() {

    k1 := Domino{5}
    k2 := Domino{7}

    fmt.Println(sepasangKartu(k1, k2))
}
```

Output:

True

Deskripsi Program:

Program ini bertujuan untuk merealisasikan aksi yang dapat dilakukan menggunakan operasi dasar mesin domino. Program dapat mencari kartu yang memiliki gambar atau nilai sisi yang sama dengan kartu tertentu melalui proses pengambilan kartu dari tumpukan secara berulang. Selain itu, program juga dapat menentukan apakah dua kartu domino membentuk pasangan dengan total nilai 12. Implementasi ini menunjukkan bagaimana operasi dasar mesin domino dapat digunakan untuk menyelesaikan permasalahan yang lebih kompleks.

3.

Source Code:

```
package main

import "fmt"
```

```

type Domino struct {
    Sisi1 int
    Sisi2 int
}

func bisaSambung(a Domino, b Domino) bool {

    return a.Sisi1 == b.Sisi1 ||
        a.Sisi1 == b.Sisi2 ||
        a.Sisi2 == b.Sisi1 ||
        a.Sisi2 == b.Sisi2
}

func main() {

    kartu1 := Domino{2, 4}
    kartu2 := Domino{4, 6}

    if bisaSambung(kartu1, kartu2) {
        fmt.Println("Kartu dapat disambung")
    } else {
        fmt.Println("Kartu tidak dapat disambung")
    }
}

```

Output:

Kartu dapat disambung

Deskripsi Program:

Program ini merupakan implementasi sederhana dari permainan Gapleh menggunakan kartu domino. Program melakukan pemeriksaan terhadap dua kartu untuk menentukan apakah kedua kartu tersebut dapat disambungkan berdasarkan kesamaan nilai pada salah satu sisinya. Melalui implementasi ini, konsep dasar permainan Gapleh dapat dipahami, yaitu menyusun kartu domino secara berurutan dengan memperhatikan kecocokan nilai antar kartu yang digunakan.

4.

Source Code:

```

package main

import "fmt"

var data string
var idx int

```

```

func start(teks string) {
    data = teks
    idx = 0
}

func maju() {
    idx++
}

func eop() bool {
    return data[idx] == '.'
}

func cc() byte {
    return data[idx]
}

func main() {

    start("BELAJAR.")

    for !eop() {
        fmt.Printf("%c ", cc())
        maju()
    }
}

```

Output:
B E L A J A R

B.1

Source Code:

```

for !eop() {
    fmt.Printf("%c", cc())
    maju()
}

```

B.2

Source Code:

```

jumlah := 0

for !eop() {
    jumlah++
    maju()
}

```

```
fmt.Println(jumlah)
```

B.3

Source Code:

```
jumlahA := 0

for !eop() {

    if cc() == 'A' {
        jumlahA++
    }

    maju()
}

fmt.Println(jumlahA)
```

B.4

Source Code:

```
frekuensi := float64(jumlahA) / float64(jumlahKarakter)

fmt.Println(frekuensi)
```

B.5

Source Code:

```
jumlahLE := 0

for i := 0; i < len(data)-1; i++ {

    if data[i] == 'L' &&
        data[i+1] == 'E' {

        jumlahLE++
    }
}

fmt.Println(jumlahLE)
```

Deskripsi Program:

Program ini mengimplementasikan mesin abstrak karakter yang digunakan untuk membaca dan mengolah rangkaian karakter hingga mencapai tanda akhir berupa titik (.). Program menyediakan operasi dasar untuk memulai pembacaan, berpindah ke karakter berikutnya, memeriksa akhir rangkaian, serta membaca karakter yang sedang aktif. Dengan operasi tersebut, program dapat melakukan berbagai pengolahan teks seperti membaca seluruh karakter, menghitung jumlah karakter, menghitung jumlah huruf "A", menentukan frekuensi kemunculan huruf "A", dan menghitung jumlah kemunculan pasangan huruf "LE" dalam suatu rangkaian karakter.

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java* (6th ed.). Wiley.

Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3rd ed.). Stanford University.

Knuth, D. E. (1997). *The Art of Computer Programming Volume 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley.